Implementación de un juego de laberinto en lenguaje ensamblador 8086

Bryan Rojas Barrantes y Carlos Andres Romero

Escuela de Informática, Facultad de Ciencias Exactas y Naturales, Universidad Nacional

Arquitectura de Computadoras - EIF205

Proyecto 2

6 de noviembre de 2026

Implementación de un juego de laberinto en lenguaje ensamblador 8086

Introducción

El Proyecto 2 del curso Arquitectura de Computadoras (EIF205) solicita desarrollar un juego 2D sencillo en lenguaje ensamblador x86 o 8086, con entrada por teclado, sistema de puntuación acumulativa, registro de marcador máximo, estados de victoria y derrota, reinicio del juego y documentación técnica del proceso (Universidad Nacional, 2026). A partir de estos requerimientos se implementó un juego de laberinto en modo texto para EMU8086.

La dinámica consiste en desplazar al jugador, representado por el símbolo @, a través de un laberinto delimitado por paredes. En cada nivel se deben recoger tres monedas, representadas por el símbolo *, antes de utilizar la salida E. El juego incluye tres niveles, un límite de movimientos por nivel, puntuación acumulativa, récord máximo durante la ejecución del programa, pausa, reinicio, regreso al menú principal y estados explícitos de Game Over y victoria.

El propósito de este informe es describir la organización del programa, el uso de memoria, los principales procedimientos y macros, las reglas de funcionamiento y los resultados de las pruebas realizadas en EMU8086. Las evidencias visuales de las pruebas se presentan en el Apéndice A.

Arquitectura del software y mapeo de memoria

Organización general

El programa utiliza el modelo de memoria small y declara una pila de 100h (256 bytes). El segmento de datos contiene tres mapas estáticos, un tablero de trabajo, variables de estado y los textos utilizados por la interfaz. El segmento de código organiza la ejecución mediante etiquetas principales, procedimientos y macros. Esta separación permite mantener de forma independiente

la carga de niveles, la presentación del tablero, la lectura del teclado, el movimiento del jugador, la actualización del marcador y el control de los estados del juego.

Cada mapa contiene 84 caracteres distribuidos en 7 filas por 12 columnas. Al iniciar un nivel, el procedimiento cargar_nivel copia el mapa correspondiente al arreglo tablero. La posición del jugador no se almacena directamente como @ dentro del tablero de trabajo, sino que se mantiene en la variable posicion; al renderizar la pantalla, mostrar_juego sustituye visualmente la celda correspondiente por el símbolo del jugador.

Tabla 1

Mapeo lógico de los principales datos del programa

Elemento	Tipo	Tamaño	Función
mapa1, mapa2, mapa3	Byte	84 bytes cada uno	Mapas estáticos de los tres niveles.
tablero	Byte	84 bytes	Copia editable del mapa activo.
posicion	Word	2 bytes	Índice actual del jugador dentro del tablero.
nivel	Byte	1 byte	Nivel actual, con valores de 1 a 3.
monedas	Byte	1 byte	Cantidad de monedas pendientes en el nivel.
movimientos	Byte	1 byte	Movimientos válidos disponibles; inicia en 45.
puntos	Word	2 bytes	Puntuación acumulada de la partida.
record	Word	2 bytes	Mayor puntuación alcanzada durante la ejecución.
estado	Byte	1 byte	0: juego activo; 1: derrota; 2: victoria.
posicion_anterior	Word	2 bytes	Posición previa usada para actualizar la pantalla.
nivel_anterior	Byte	1 byte	Permite detectar un cambio de nivel.

Nota. El mapeo es lógico y describe el tamaño y la función de los datos; las direcciones físicas son asignadas por el ensamblador durante la construcción del programa.

Manejo de pila y registros

Los procedimientos que utilizan registros de propósito general preservan los registros necesarios mediante instrucciones PUSH y POP. Por ejemplo, cargar_nivel conserva AX, CX, SI y DI; mostrar_juego conserva AX, BX, CX, DX y SI; mover_jugador conserva AX, BX y DX. Esta práctica evita que una rutina modifique de forma accidental valores utilizados por la rutina que la invocó.

Descripción de módulos y rutinas

La ejecución inicia preparando el segmento de datos y mostrando el menú principal. A partir de ese punto, el programa cambia entre menú, partida activa, pausa y estados finales según la tecla introducida por el usuario. La lógica se distribuye en procedimientos con responsabilidades concretas.

Tabla 2

Principales procedimientos y funciones del programa

Rutina	Responsabilidad principal
limpiar_pantalla	Restablece el modo texto mediante INT 10h y limpia la pantalla.
escribir	Muestra cadenas terminadas en $ mediante INT 21h, función 09h.
leer_tecla	Lee una tecla sin eco y convierte letras mayúsculas a minúsculas.
cargar_nivel	Selecciona el mapa, lo copia a tablero y reinicia monedas y movimientos.
mostrar_juego	Dibuja título, marcador, laberinto, jugador y ayuda de controles.
mostrar_marcador	Presenta nivel, puntos, récord, movimientos y monedas restantes.
posicionar_cursor	Ubica el cursor en una fila y columna determinadas.
pintar_una_celda	Actualiza una celda específica del tablero en pantalla.
actualizar_movimiento	Redibuja únicamente la posición anterior, la nueva posición y el marcador.
imprimir_caracter	Muestra un carácter mediante INT 21h, función 02h.
imprimir_numero	Convierte el valor de AX a decimal y lo imprime dígito por dígito.
mover_jugador	Procesa W/A/S/D, colisiones, monedas, salida, cambio de nivel y derrota.
actualizar_record	Compara la puntuación actual con el récord y conserva el mayor valor.

Macros implementadas

Para reducir instrucciones repetidas de salida se incorporaron las macros MOSTRAR_TEXTO y SALTO_LINEA. MOSTRAR_TEXTO recibe el nombre de una cadena, carga su dirección en DX y llama al procedimiento escribir. SALTO_LINEA imprime los códigos 13 y 10 mediante imprimir_caracter. Estas macros simplifican operaciones frecuentes de interfaz sin modificar la lógica de movimiento, colisiones, puntuación o estados del juego.

Reglas y estados del juego

El jugador se desplaza con W, A, S y D. Cada movimiento válido modifica el índice de posicion: subir resta 12, bajar suma 12, izquierda resta 1 y derecha suma 1. Antes de confirmar el movimiento se verifica que el índice permanezca dentro del tablero y que la celda de destino no contenga una pared (#). Los intentos contra una pared no cambian la posición ni consumen movimientos.

Por tratarse de un laberinto de cuadrícula en modo texto, el movimiento se modela de forma discreta, una celda por entrada válida. En este género no se utiliza aceleración continua; la mecánica física pertinente se concentra en los límites del tablero, la detección de paredes y la validación de la salida.

Cada nivel contiene tres monedas. Al entrar en una celda con *, la moneda se elimina del tablero, monedas disminuye en una unidad y puntos aumenta en 10. La salida E se mantiene bloqueada mientras monedas sea diferente de cero. Al entrar en la salida después de recoger las tres monedas se agregan 50 puntos. Si el nivel actual es 1 o 2, se carga el nivel siguiente con 45 movimientos nuevos; si se completa el nivel 3, estado cambia a 2 y se muestra la victoria.

La derrota ocurre cuando movimientos llega a cero después de un movimiento válido que no completa el objetivo. En ese caso estado cambia a 1 y se presenta el mensaje de Game Over.

La tecla P activa una pausa que ignora otras entradas hasta volver a presionar P. La tecla R inicia una nueva partida sin cerrar el emulador; reinicia puntos, nivel y estado, pero conserva el récord. La tecla Q regresa al menú principal.

Bitácora de pruebas

Las pruebas funcionales se realizaron en EMU8086 y se orientaron a comprobar movimiento, colisiones, recolección de objetos, restricciones de salida, transición entre niveles, pausa, reinicio, menú, Game Over, puntuación y victoria. Las capturas correspondientes se muestran en el Apéndice A.

Tabla 3

Resultados de las pruebas funcionales realizadas en EMU8086

Caso	Acción	Resultado esperado	Resultado observado	Evidencia
T01	Iniciar una partida desde el menú.	Cargar Nivel 1 con controles y marcador visibles.	Correcto. El Nivel 1 se cargó con 45 movimientos y 3 monedas.	Figura A1
T02	Intentar atravesar una pared (#).	Mantener la posición y no consumir movimientos.	Correcto. El jugador no atravesó la pared y el contador permaneció en 30.	Figura A2
T03	Recoger las tres monedas del Nivel 1.	Sumar 10 puntos por moneda y reducir el contador a 0.	Correcto. Se obtuvieron 30 puntos y 0 monedas restantes.	Figura A3
T04	Llegar a E con monedas pendientes.	Bloquear el cambio de nivel.	Correcto. En el Nivel 2 la salida no permitió avanzar con 3 monedas pendientes.	Figura A4
T05	Recoger monedas y utilizar E.	Avanzar de nivel y añadir el bono de 50 puntos.	Correcto. Se pasó al Nivel 2 con 80 puntos y 45 movimientos.	Figura A5
T06	Presionar P, intentar moverse y presionar P nuevamente.	Pausar, impedir movimiento y reanudar conservando el estado.	Correcto. Durante la pausa no hubo movimiento ni cambios en el marcador.	Figura A6
T07	Presionar R durante la partida.	Reiniciar sin cerrar EMU8086 y conservar el récord.	Correcto. Regresó al Nivel 1 con puntos 0, movimientos 45, monedas 3 y récord 80.	Figura A7
T08	Agotar todos los movimientos válidos.	Mostrar Game Over y opciones de reinicio o menú.	Correcto. Con movimientos en 0 apareció el mensaje de Game Over.	Figura A8

Caso	Acción	Resultado esperado	Resultado observado	Evidencia
T09	Presionar Q y comenzar nuevamente desde el menú.	Volver al menú y permitir una nueva partida.	Correcto. El menú reapareció y la nueva partida mantuvo el récord alcanzado.	Figura A9
T10	Completar los tres niveles.	Mostrar victoria y actualizar el récord.	Correcto. Se mostró GANASTE con 240 puntos, récord 240 y 7 movimientos restantes.	Figura A10
T11	Recompilar la versión final con macros y repetir movimiento, moneda, pausa y reinicio.	Mantener el comportamiento funcional después de incorporar MOSTRAR_TEXTO y SALTO_LINEA.	Correcto. Se recogió una moneda, la pausa bloqueó el movimiento y R restableció el Nivel 1 con 45 movimientos y 3 monedas, conservando el récord.	Figura A11

Registro de errores y soluciones

Durante la sesión de pruebas documentada no se detectaron errores funcionales en la lógica evaluada. Por esa razón, no fue necesario aplicar correcciones a movimientos, colisiones, puntuación, estados o transición entre niveles durante dichas pruebas.

Tabla 4

Registro de errores durante la validación funcional

Error detectado	Impacto	Solución aplicada	Verificación
No se detectaron errores funcionales durante las pruebas ejecutadas.	Sin impacto funcional en la versión probada.	No fue necesario modificar la lógica del juego durante esta sesión.	Las pruebas T01-T11 se completaron satisfactoriamente.

Validación de la versión final

Después de incorporar las macros MOSTRAR_TEXTO y SALTO_LINEA, el archivo LABERINTO.ASM se compiló nuevamente en EMU8086. Se repitieron pruebas de movimiento, recolección de moneda, pausa, reanudación y reinicio para comprobar que la refactorización de la salida de texto no alterara la lógica del juego. La prueba fue satisfactoria: la moneda actualizó puntos y récord, la pausa impidió el movimiento y el reinicio restauró los valores iniciales del Nivel 1 manteniendo el récord. Esta verificación corresponde al caso T11 y se documenta en la Figura A11.

Control de versiones y documentación

El proyecto utiliza Git para mantener un historial progresivo y distribuido entre los integrantes. El repositorio incluye el código fuente LABERINTO.ASM y un README.md con la descripción del juego, requisitos, controles e instrucciones de compilación y ejecución en EMU8086. El historial registra contribuciones realizadas por Bryan Rojas Barrantes y Carlos Andres Romero, en concordancia con el requisito de control de versiones del proyecto. Repositorio del proyecto.

Conclusiones

La implementación permitió comprobar que un juego de laberinto puede estructurarse de forma modular en ensamblador 8086 utilizando procedimientos, macros, variables de estado y un tablero representado en memoria. La separación entre carga de niveles, entrada de teclado, renderizado, movimiento, actualización del marcador y control de estados facilita la comprensión del flujo del programa y reduce la dependencia entre tareas.

Las pruebas realizadas confirmaron el funcionamiento de las principales reglas del juego: detección de paredes, recolección de monedas, bloqueo de la salida, puntuación acumulativa, cambio de nivel, pausa, reinicio, regreso al menú, Game Over, victoria y conservación del récord dentro de la misma ejecución. Las evidencias muestran además que el límite de movimientos influye directamente en la estrategia del jugador, ya que obliga a completar cada nivel antes de agotar el contador disponible.

El uso de EMU8086 permitió observar de manera directa el comportamiento del programa en modo texto y verificar la interacción entre las rutinas. La recompilación de la versión final con macros confirmó que la reorganización de las operaciones repetitivas de salida no modificó el comportamiento funcional. Además, el uso de Git y del README permitió complementar la implementación con control de versiones y documentación de ejecución. Como resultado, se obtuvo una implementación sencilla pero completa que responde a los requerimientos funcionales planteados para el proyecto y que puede explicarse a partir de sus módulos principales, su organización de memoria y sus estados de ejecución.

Referencias

Universidad Nacional. (2026). *Proyecto 2: Proyecto de implementación de un juego en lenguaje ensamblador* [Guía de proyecto]. Escuela de Informática, Facultad de Ciencias Exactas y Naturales.

Apéndice A

Evidencias de pruebas en EMU8086

Las siguientes figuras corresponden a las capturas obtenidas durante la ejecución de las pruebas funcionales descritas en la Tabla 3.

Figura A1

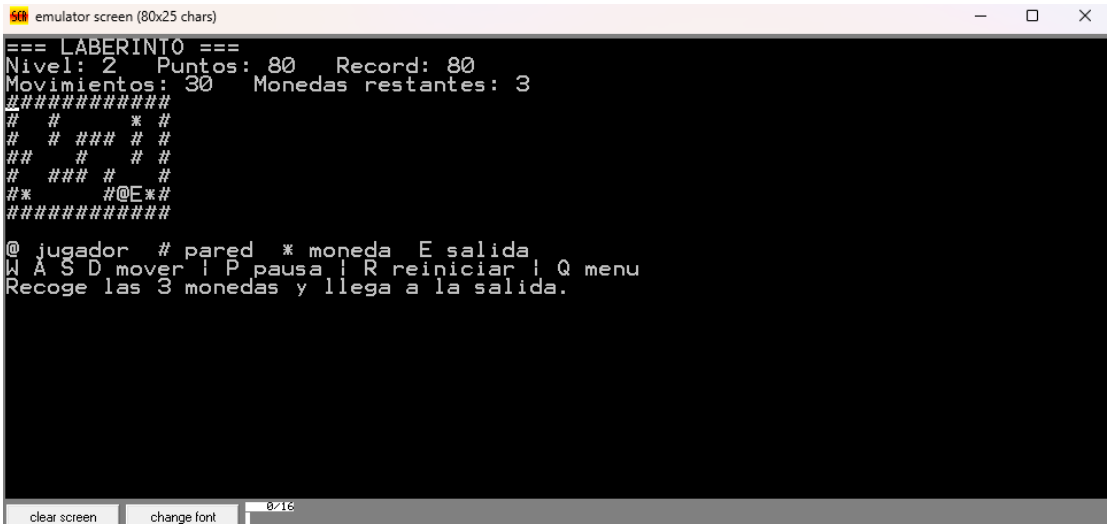
Inicio de una partida en el Nivel 1



Nota. El juego muestra el tablero, el marcador y los controles. La captura corresponde a una nueva partida iniciada desde el menú durante la sesión de pruebas.

Figura A2

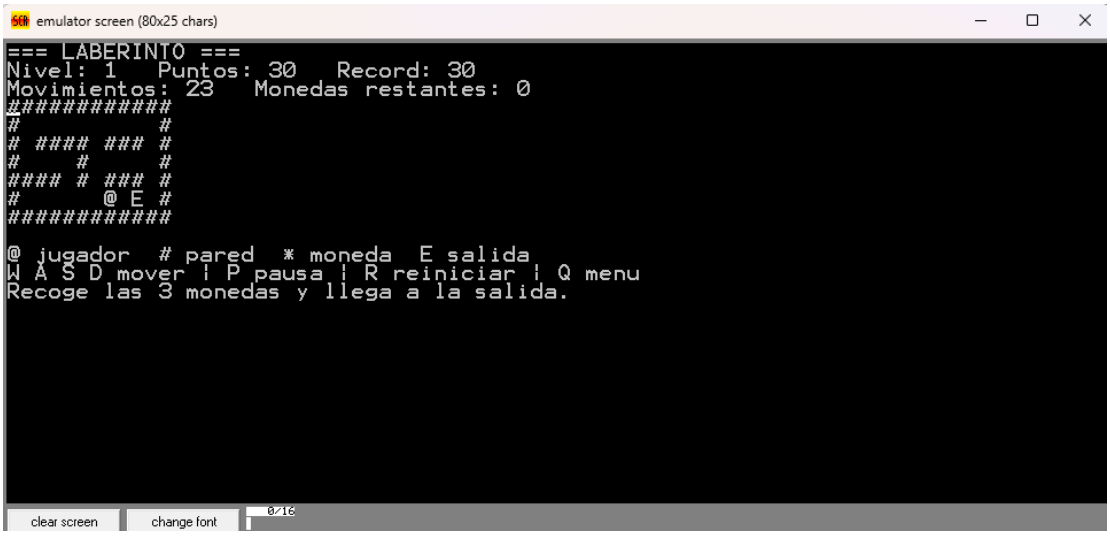
Comprobación de colisión con pared



Nota. El jugador permanece en la misma posición y el contador de movimientos se mantiene en 30 al intentar desplazarse hacia una pared.

Figura A3

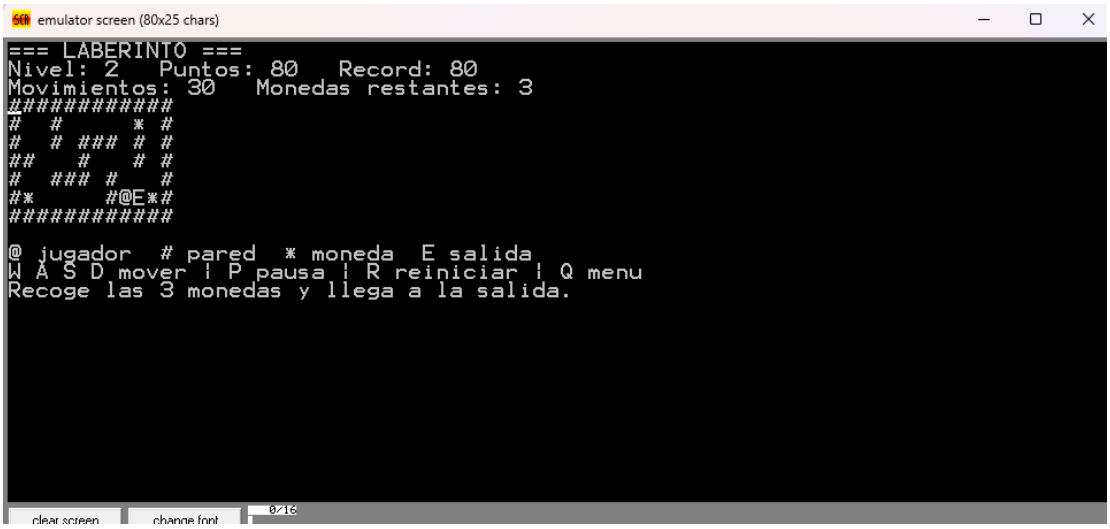
Recolección de las tres monedas del Nivel 1



Nota. El marcador muestra 30 puntos, 0 monedas restantes y 23 movimientos disponibles antes de utilizar la salida.

Figura A4

Salida bloqueada con monedas pendientes



Nota. En el Nivel 2 el jugador alcanza la zona de salida, pero no puede avanzar mientras el marcador indica 3 monedas restantes.

Figura A5

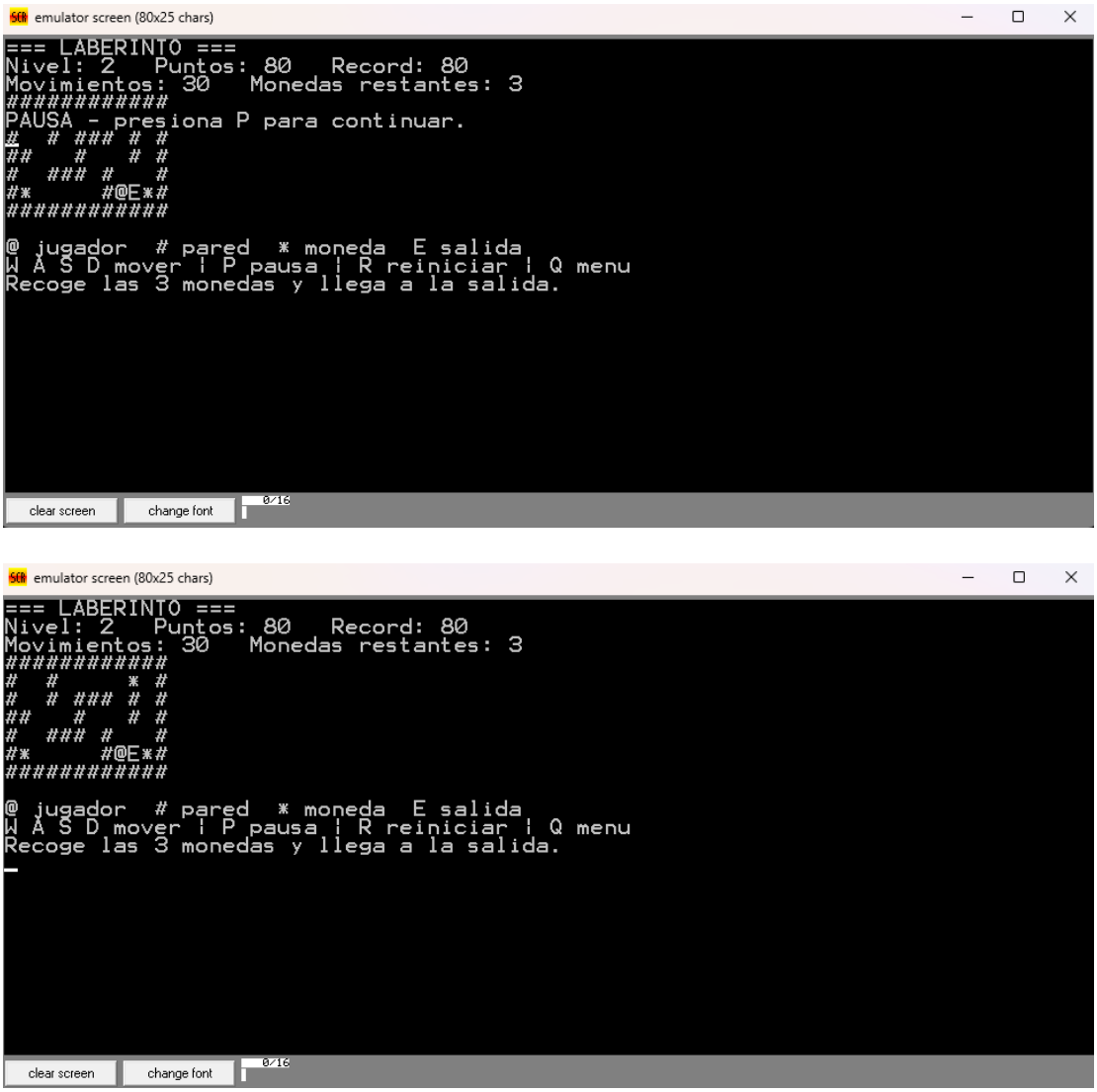
Transición correcta al Nivel 2



Nota. Después de completar el Nivel 1, el programa carga el Nivel 2 con 45 movimientos, 3 monedas y 80 puntos acumulados.

Figura A6

Pausa y reanudación de la partida



Nota. La primera captura muestra el mensaje de pausa. Durante este estado se intentó mover al jugador sin éxito. La segunda captura muestra la reanudación con el mismo marcador y posición.

Figura A7

Reinicio de la partida con la tecla R



Nota. El reinicio devuelve el juego al Nivel 1 con puntos en 0, 45 movimientos y 3 monedas, mientras conserva el récord de 80 puntos.

Figura A8

Game Over por agotamiento de movimientos



Nota. Al llegar a 0 movimientos se muestra el estado de derrota y las opciones para volver a jugar con R o regresar al menú con Q.

Figura A9

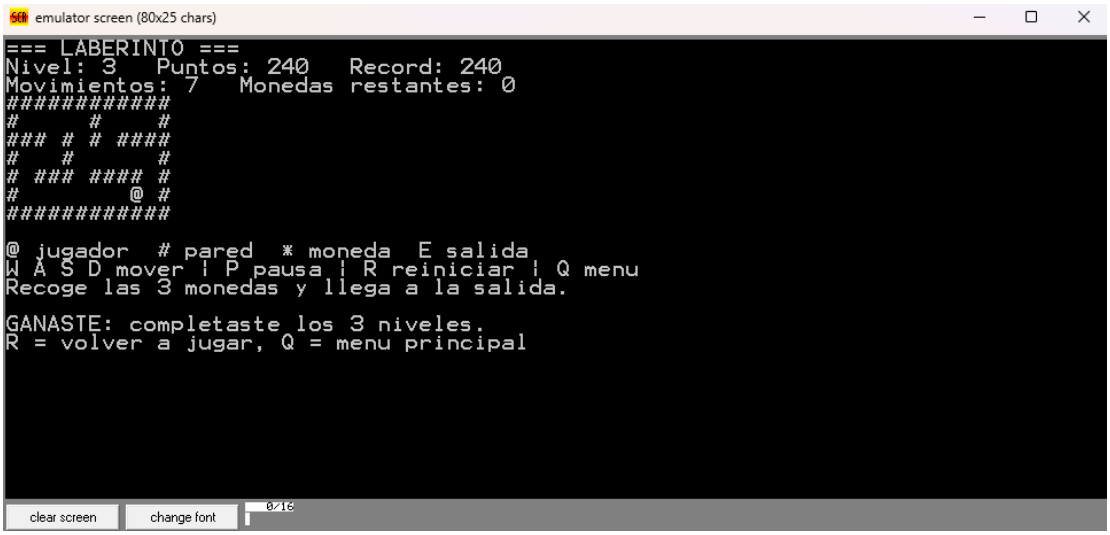
Regreso al menú principal con la tecla Q



Nota. El programa regresa al menú principal sin cerrar el emulador y permite seleccionar nuevamente la opción de jugar.

Figura A10

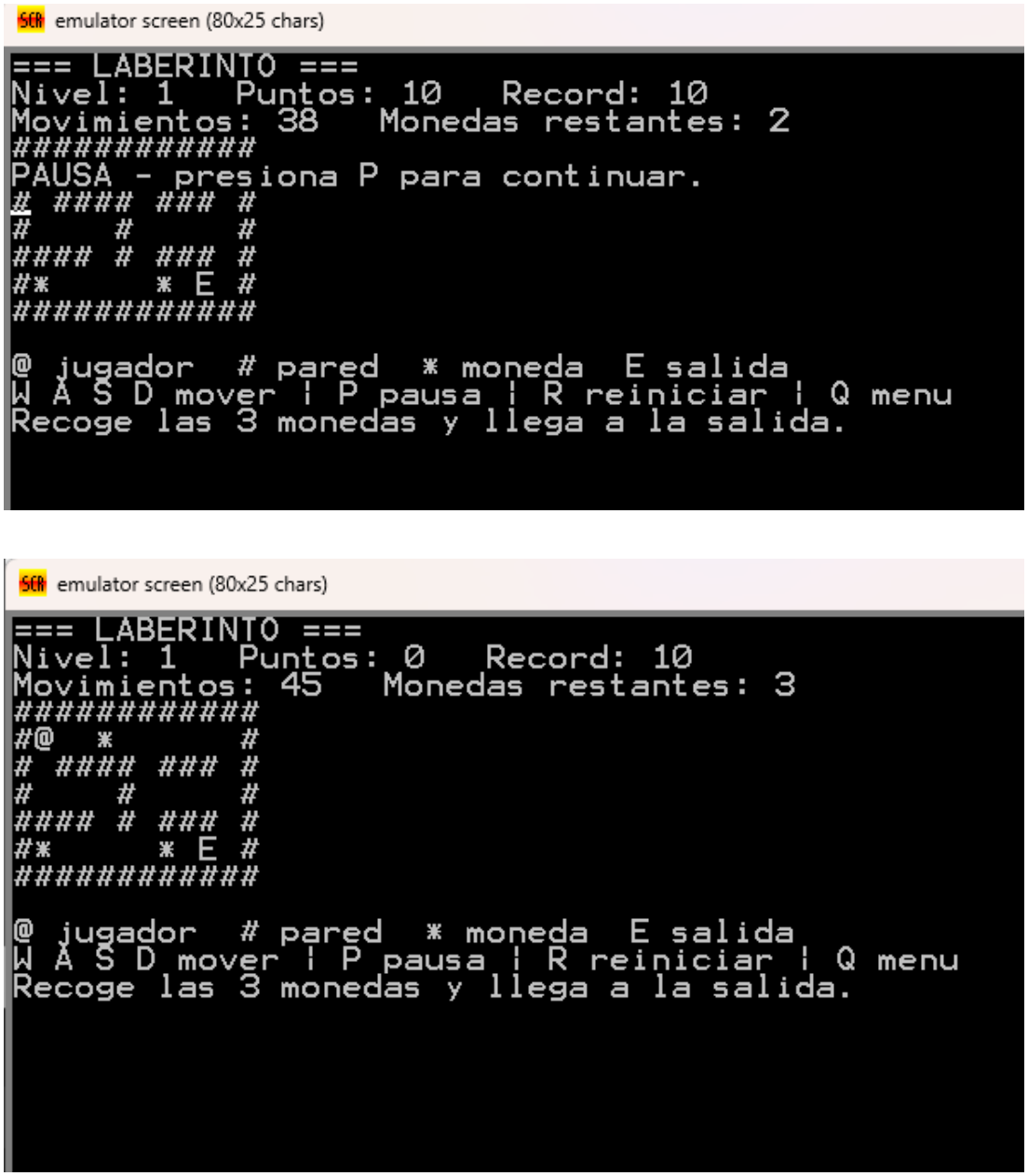
Victoria después de completar los tres niveles



Nota. La pantalla final muestra 240 puntos, récord de 240, 0 monedas restantes y el mensaje de victoria tras completar el Nivel 3.

Figura A11

Validación de regresión de la versión final con macros



Nota. La primera captura corresponde a la versión final con macros después de recoger una moneda y activar la pausa; se observan 10 puntos, récord 10, 38 movimientos y 2 monedas restantes. La segunda captura muestra el resultado de presionar R: el juego vuelve al Nivel 1 con 0 puntos, 45 movimientos y 3 monedas, mientras conserva el récord de 10.